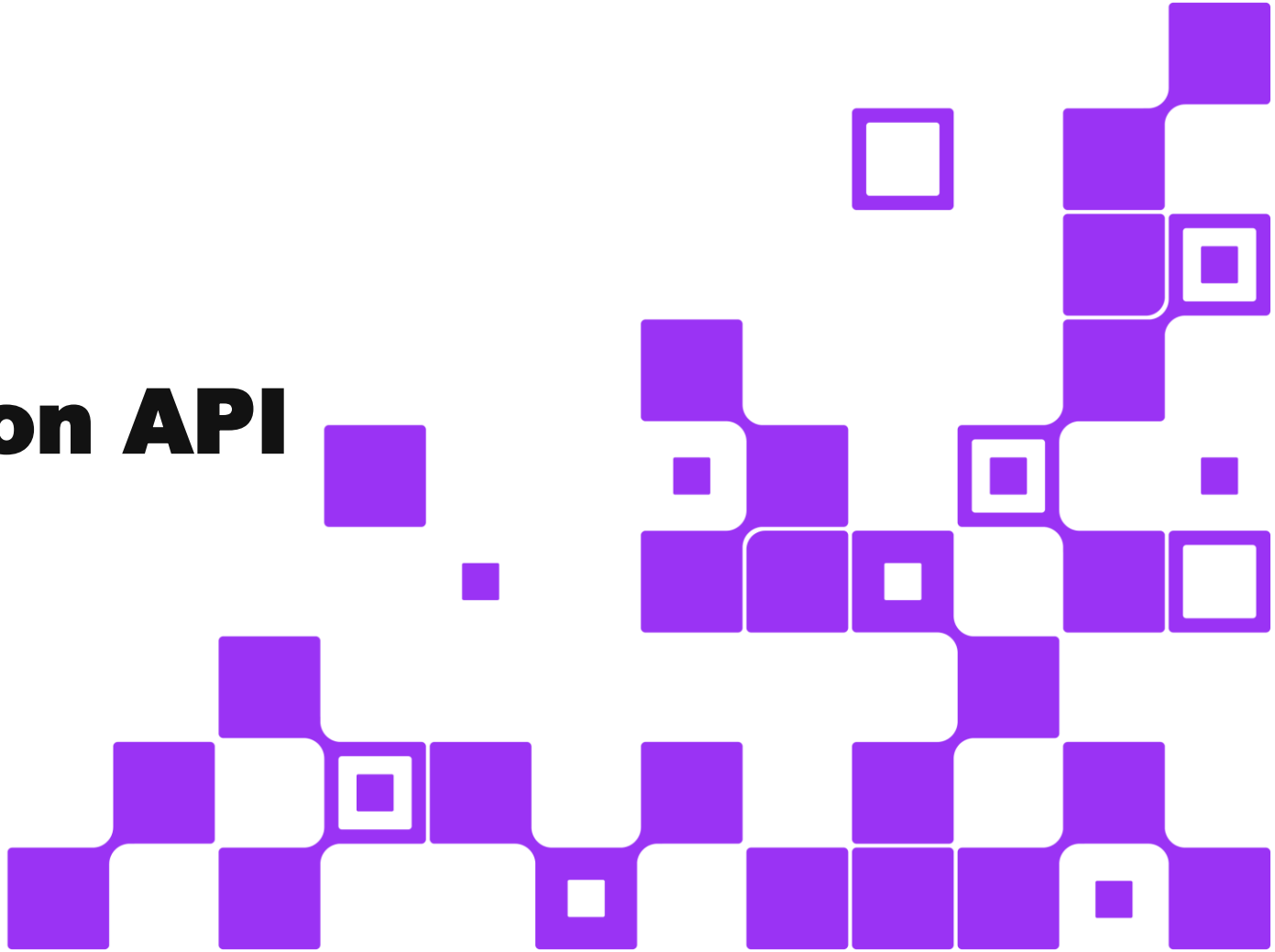
IThub

Java Reflection API



IThub

Reflection API — это механизм Java, который позволяет программе исследовать и изменять собственную структуру во время выполнения.

- узнать имя класса объекта
- получить список полей
- найти методы
- вызвать метод динамически
- создать объект
- изменить доступ к приватным членам класса

Обычный Java-код vs Reflection

IThub

java код

```
User user = new User();

user.setName("Alex");
user.print();
```

В привычном нам java коде компилятор знает всё заранее. Связи определяются во время компиляции.

С Reflection класс может быть неизвестен заранее. Связь создаётся во время выполнения.

Reflection

```
Class<?> clazz =
Class.forName("com.example.User");

Object obj =
    clazz.getDeclaredConstructor()
        .newInstance();
```

java.lang.reflect

IThub

Reflection API находится в пакете: java.lang.reflect
Основные классы:

Класс	Назначение	Класс
Class	описание класса	Class
Field	поле класса	Field
Method	метод класса	Method
Constructor	конструктор	Constructor
Modifier	модификаторы доступа	Modifier
Array	работа с массивами	Array

Класс Class — основа Reflection. Каждый объект в Java связан с объектом типа Class

Три способа получить объект Class

IThub

Через объект

```
User user = new User();

Class<?> clazz = user.getClass();
```

Через имя класса

```
Class<?> clazz =
    Class.forName(
        "com.example.User"
    );
```

Через .class

```
Class<?> clazz = User.class;
```

Получение информации о классе

IThub

- clazz.getSimpleName()
- clazz.getPackage() - возвращает пакет для этого класса
- clazz.getSuperclass() - возвращает объект Class, представляющий суперкласс сущности (класса, интерфейса, примитивного типа или void), которую представляет данный объект Class
- clazz.getInterfaces() - непосредственно реализованные интерфейсы

```
Class<?> clazz = User.class;

System.out.println(
    clazz.getName()
);
```

```
com.example.User
```

Получение полей класса

IThub

```
class User {

    private String name;
    public int age;

}
```

getDeclaredFields() возвращает массив объектов **Field**, представляющих все поля, объявленные в классе или интерфейсе, который описывается данным объектом.
Сюда входят поля с модификаторами доступа public, protected, private и поля с доступом по умолчанию, но не включаются унаследованные поля.

```
Field[] fields =
    User.class.getDeclaredFields();

for(Field field : fields){
    System.out.println(
        field.getName()
    );
}
```

```
name
age
```

Работа с приватными полями

IThub

```
public class Task
    private String
nameTask;
    private String opisTask;
    private String dateTask;
```

```
Task task = new Task();
task.nameTask = “New name";
```

Reflection:

```
Task taskExample = new Task("name", "description", "2026-09-21");
System.out.println(taskExample.getNameTask());

Field field = Task.class.getDeclaredField("nameTask");
field.setAccessible(true);
field.set(taskExample,"NEW NAME");

System.out.println(taskExample.getNameTask());
```

```
name
NEW NAME
```

Получение методов класса

IThub

```
Method[] methods = Task.class.getMethods();

    for(Method m : methods){
        System.out.println(m.getName());
    }
```

getMethods() возвращает массив, содержащий объекты **Method**, отражающие все открытые (public) методы класса или интерфейса, представленного данным объектом, включая методы, объявленные в этом классе или интерфейсе, а также унаследованные от суперклассов и суперинтерфейсов.

```
setDateTask
getNameTask
setNameTask
setOpisTask
getOpisTask
getDateTask
equals
toString
hashCode
getClass
notify
notifyAll
wait
wait
wait
```

Вызов метода через Reflection

IThub

Обычный вызов:

```
Task taskExample = new Task("name", "description", "2026-09-21");
taskExample.getNameTask();
```

Reflection:

```
Method method = Task.class.getMethod("getNameTask");
String name = (String) method.invoke(taskExample);
System.out.println(name);
```

```
name
```

Доступ к приватным методам

IThub

- getMethods() – только public методы
- getDeclaredMethods() - включает private, protected и package-private методы

```
class User {

    private void sayHello() {
        System.out.println("Hello!");
    }
}
```

```
user.sayHello();
```

Доступ к приватным методам

IThub

```
class User {

    private void sayHello() {
        System.out.println("Hello!");
    }
}
```

Reflection:

```
User user = new User();

Method method =

User.class.getDeclaredMethod("sayHello");

method.setAccessible(true);

method.invoke(user);
```

Методы с параметром

IThub

```
class Calculator {

    private int sum(int a, int b) {
        return a + b;
    }
}
```

Reflection:

```
Method method = Calculator.class
        .getDeclaredMethod(
            "sum",
            int.class,
            int.class
        );

method.setAccessible(true);

Object result = method.invoke(calculator, 5, 3);
```

Создание объекта через Reflection

IThub

Обычный вызов:

```
Task taskExample = new Task("name", "description", "2026-09-21");
```

Reflection:

```
Class<?> clazz = Task.class;

    Task taskReflectionWithoutParameters = (Task) clazz
            .getConstructor()
            .newInstance();

    Task taskReflectionWithParameters = (Task) clazz
            .getConstructor(String.class, String.class, String.class)
            .newInstance("name", "description", "2026-09-21");
```

IThub

Где используется Reflection

Spring Framework

Spring создаёт объекты автоматически

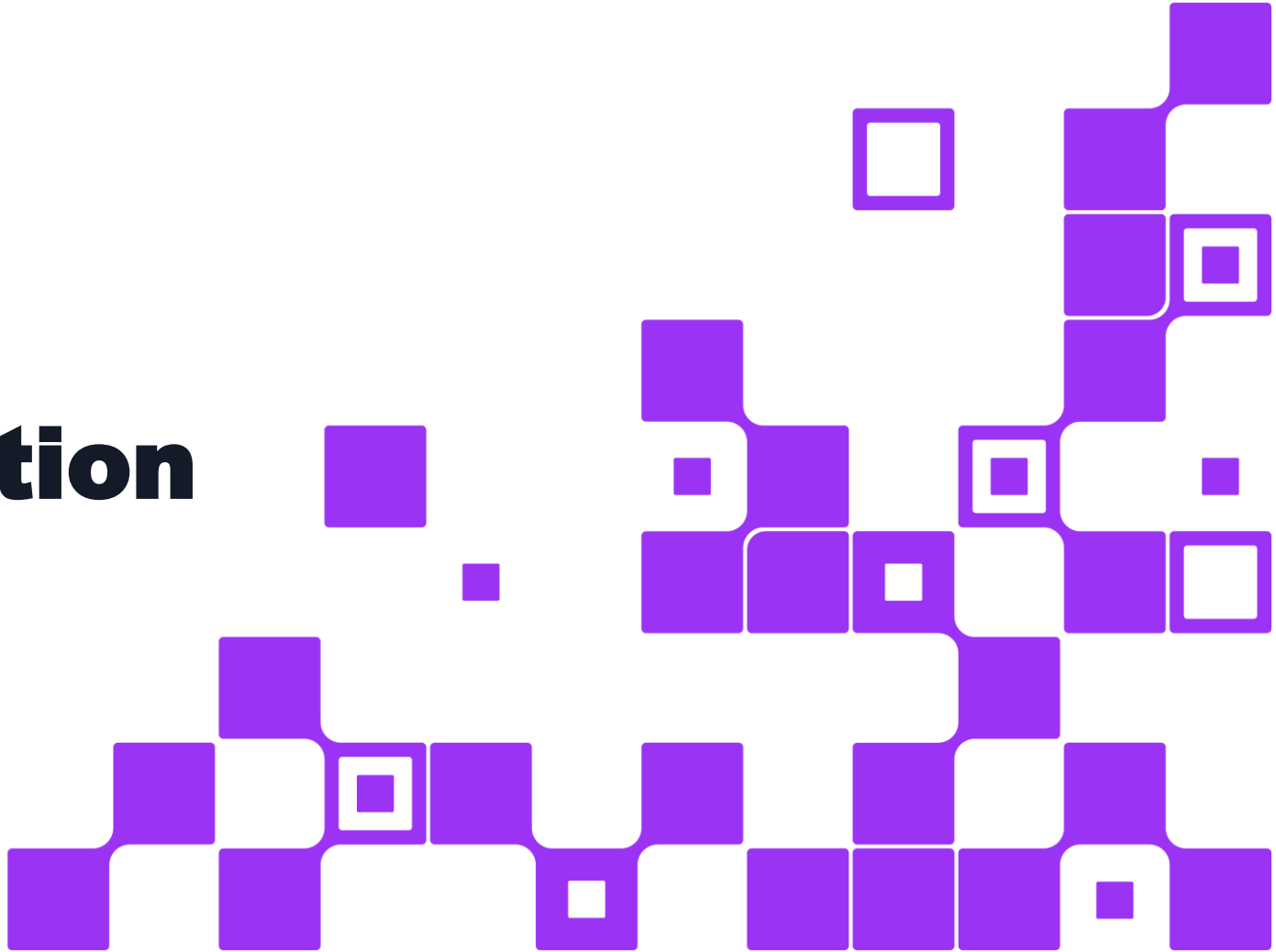
- находит класс
- анализирует аннотации
- создаёт объект
- внедряет зависимости

Hibernate

- анализирует поля
- создаёт SQL
- заполняет объекты из БД

IThub

Annotation



IThub

С помощью Java рефлексии мы можем обрабатывать аннотации в рантайме.

Аннотации - это форма метаданных. Они предоставляют информацию о программе, при том сами частью программы не являются.

Информация для компилятора

Обработка во время компиляции и развертывания

Обработка во время выполнения

Примеры аннотаций

IThub

@Override

указывает компилятору, что текущий метод переопределяет метод из суперкласса.

@Deprecated

помечает элемент (класс, метод, поле) как устаревший

@Test

говорит среде тестирования, что конкретный метод является самостоятельным тестом, который нужно запустить.

@FunctionalInterface

указывает, что интерфейс предназначен для использования в качестве функционального интерфейса (т.е. он должен иметь ровно один абстрактный метод).

Создаём свою аннотацию

IThub

Аннотации создаются через ключевое слово @interface:

```
public @interface Info {

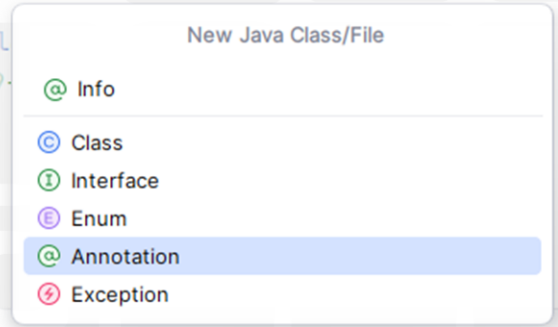
    String author();

    String description();

}
```

Но пока Java не знает, где эту аннотацию можно применять и сколько она должна жить.

В IntelliJ idea:
new Java class → Annotaion



```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Info {

    String author();

    String description();
}
```

IThub

@Retention - определяет время жизни аннотации

- **RetentionPolicy.RUNTIME** - аннотация будет доступна во время выполнения программы (это обязательно для Reflection)

@Target - где можно использовать аннотацию

- **ElementType.TYPE** – только для классов

@Retention

IThub

RetentionPolicy.SOURCE: аннотация существует только в исходном коде и отбрасывается компилятором (для инструментов анализа кода или кодогенерации)

RetentionPolicy.CLASS: аннотация сохраняется в файле .class (в байт-коде), но недоступна во время выполнения.

RetentionPolicy.RUNTIME: аннотация сохраняется в файле .class, но в отличие от предыдущего варианта аннотация доступна во время выполнения через рефлексию (например, Spring или JUnit)

@Target

IThub

ElementType.ANNOTATION_TYPE	объявления типов аннотаций
ElementType.MODULE	модули
ElementType.PACKAGE	пакеты
ElementType.TYPE	классы (включая перечисления) и интерфейсы (включая типы аннотаций)
ElementType.METHOD	методы
ElementType.CONSTRUCTOR	конструкторы
ElementType.FIELD	поля (включая константы перечисления)
ElementType.PARAMETER	параметры метода или конструктора

Используем нашу аннотацию

IThub

```
@Info(
        author = "Alex",
        description = "User entity"
)
public class User {
    private String name;

    public void hello() {
        System.out.println("Hello");
    }
}
```

Теперь класс содержит метаданные:

User.class
- name
- hello()
- @Info
  - author = Alex
  - description = User entity

Читаем аннотацию через Reflection

IThub

```
public class AnnotationReader {

    public static void main(String[] args) {
        Class<User> clazz = User.class;

        Info info = clazz.getAnnotation(Info.class);

        System.out.println(info.author());
        System.out.println(info.description());
    }
}
```

Проверяем наличие аннотации

IThub

Перед получением лучше проверить:

```
Class<User> clazz = User.class;

    if (clazz.isAnnotationPresent(Info.class)) {

        Info info = clazz.getAnnotation(Info.class);

        System.out.println(info.author());
    }
```

Аннотация для методов

IThub

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Execute {

}
```

IThub

```
public class Service {
    @Execute
    public void start() {

        System.out.println(
                "Service started"
        );
    }


    public void stop() {
        System.out.println(
                "Service stopped"
        );
    }
}
```

```
public class Runner {

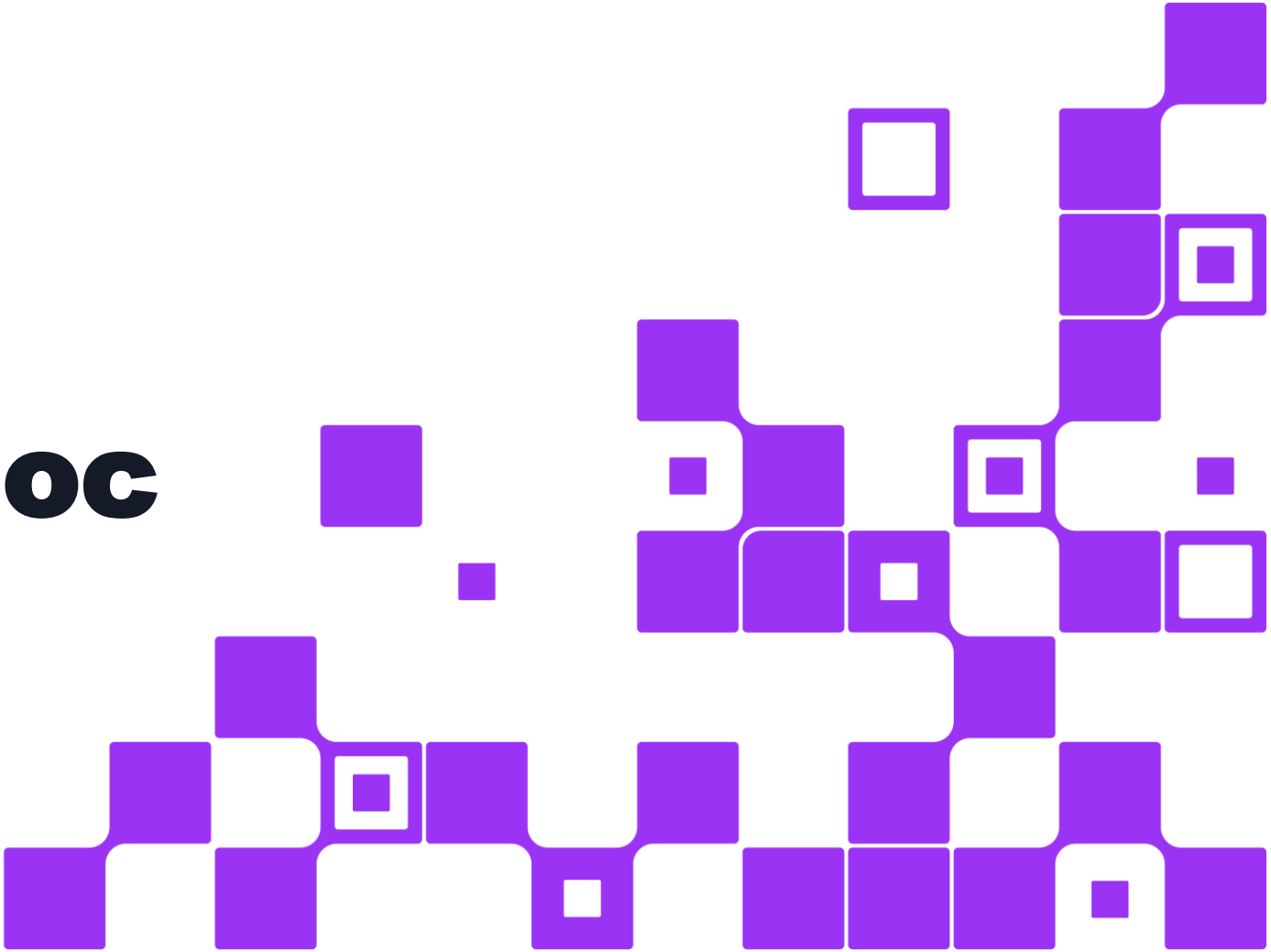
    public static void main(String[] args) throws … {
        Service service = new Service();

        Method[] methods = Service.class.getDeclaredMethods();

        for (Method method: methods) {
            if (method.isAnnotationPresent(Execute.class)) {
                method.invoke(service);
            }
        }
    }
}
```

IThub

JavaDoc



IThub

JavaDoc — это стандартный инструмент Java для создания документации из исходного кода.

Основная идея — документация пишется не отдельно от программы, а прямо в исходном коде.

можно документировать классы и интерфейсы

методы, поля, параметры методов, возвращаемые значения

исключения

IThub

JavaDoc — это специальный формат комментариев, который Java может преобразовать в HTML-документацию.

```
/**
 * Описание класса.
 */
public class User {
}
```

В отличие от обычного комментария //, JavaDoc-комментарий начинается с /** и заканчивается */.

IThub

Зачем нужен JavaDoc?

- быстро понять назначение класса или метода
- разобраться с параметрами метода
- узнать, что возвращает метод
- увидеть возможные исключения
- автоматически создавать документацию проекта
- упростить работу других разработчиков с кодом

Документирование кода

IThub

```
/**
 * Вычисляет сумму двух чисел.
 *
 * @param a первое число
 * @param b второе число
 * @return сумму чисел a и b
 */
public int sum(int a, int b) {
    return a + b;
}
```

основной текст описывает назначение метода;
@param описывает параметры;
@return описывает возвращаемое значение.

Основные JavaDoc-теги

IThub

Тег	Назначение
@param	описание параметра
@return	описание возвращаемого значения
@throws	описание исключения
@see	ссылка на связанный класс или метод
@author	автор
@version	версия
@since	версия, с которой появился элемент
@deprecated	элемент устарел и не рекомендуется к использованию

```java
/**
 * Делит одно число на другое.
 *
 * @param a делимое
 * @param b делитель
 * @return результат деления
 * @throws ArithmeticException если b равно 0
 */
public int divide(int a, int b) {
    return a / b;
}
```

Документирование класса

IThub

```
/**
 * Представляет пользователя системы.
 *
 * <p>Хранит имя и возраст пользователя.
 *
 * @author Alex
 * @version 1.0
 */
public class User {

    private String name;
    private int age;
}
```

JavaDoc можно использовать не только для методов, но и для целых классов

JavaDoc поддерживает HTML

IThub

```
/**
 * Создаёт нового пользователя.
 *
 * <p>Имя пользователя должно быть указано обязательно.
 *
 * <b>Пример:</b>
 * <pre>
 * User user = new User("Alex");
 * </pre>
 *
 * @param name имя пользователя
 */
public User(String name) {
    this.name = name;
}
```

В результате документация будет содержать форматированный текст и пример использования.

Ссылки между классами

IThub

```
/**
 * Выполняет регистрацию пользователя.
 *
 * Подробнее см. {@link User}.
 *
 * @param user пользователь для регистрации
 */
public void register(User user) {
    // ...
}
```

С помощью {@link} можно создавать ссылки на другие элементы программы.

Как создать документацию

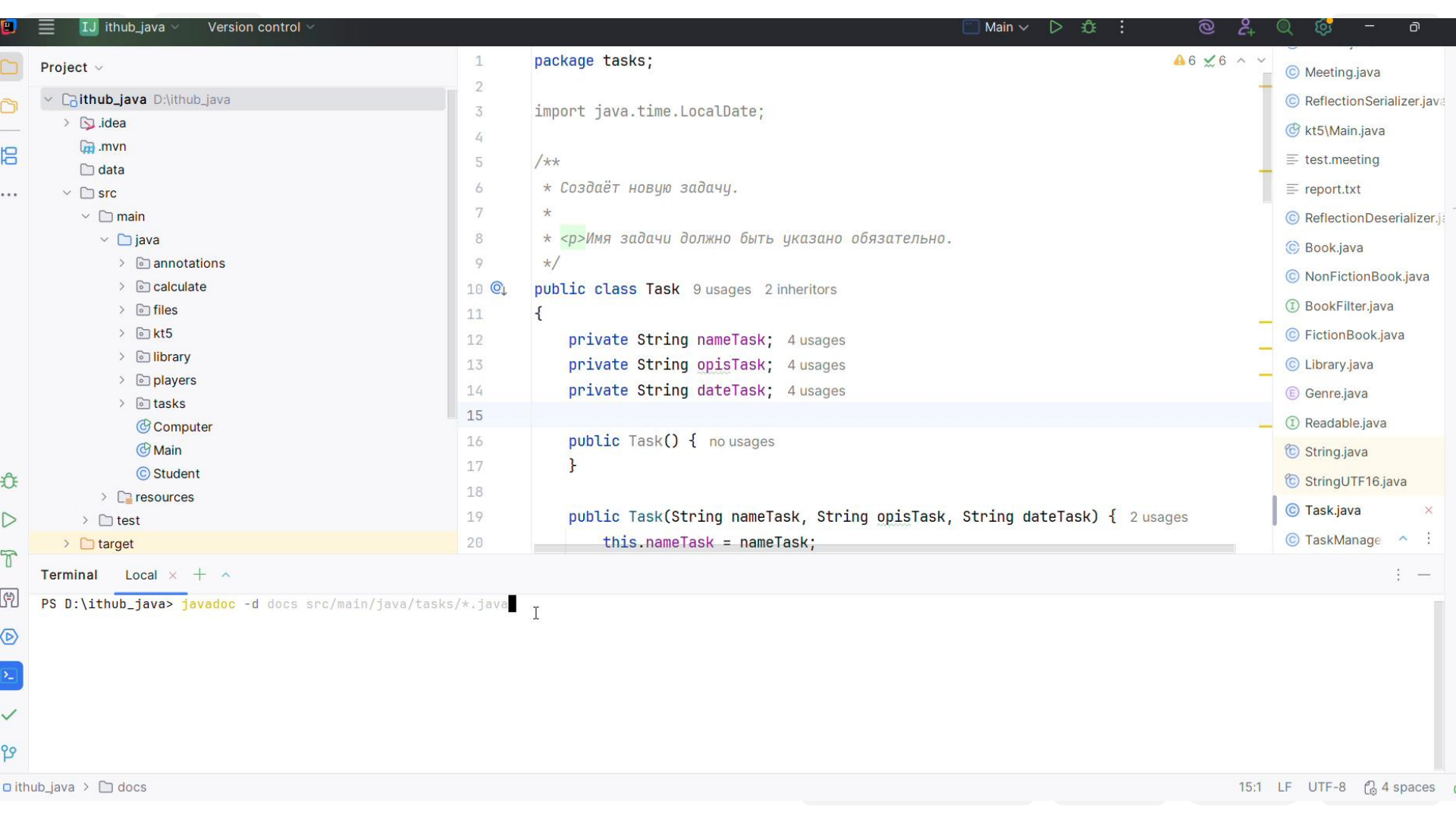
IThub

JavaDoc можно запустить из командной строки

```
javadoc -d docs src/main/java/tasks/*.java
```

Параметр -d docs указывает директорию, куда будет помещена документация.

После выполнения команды создаётся набор HTML-файлов.

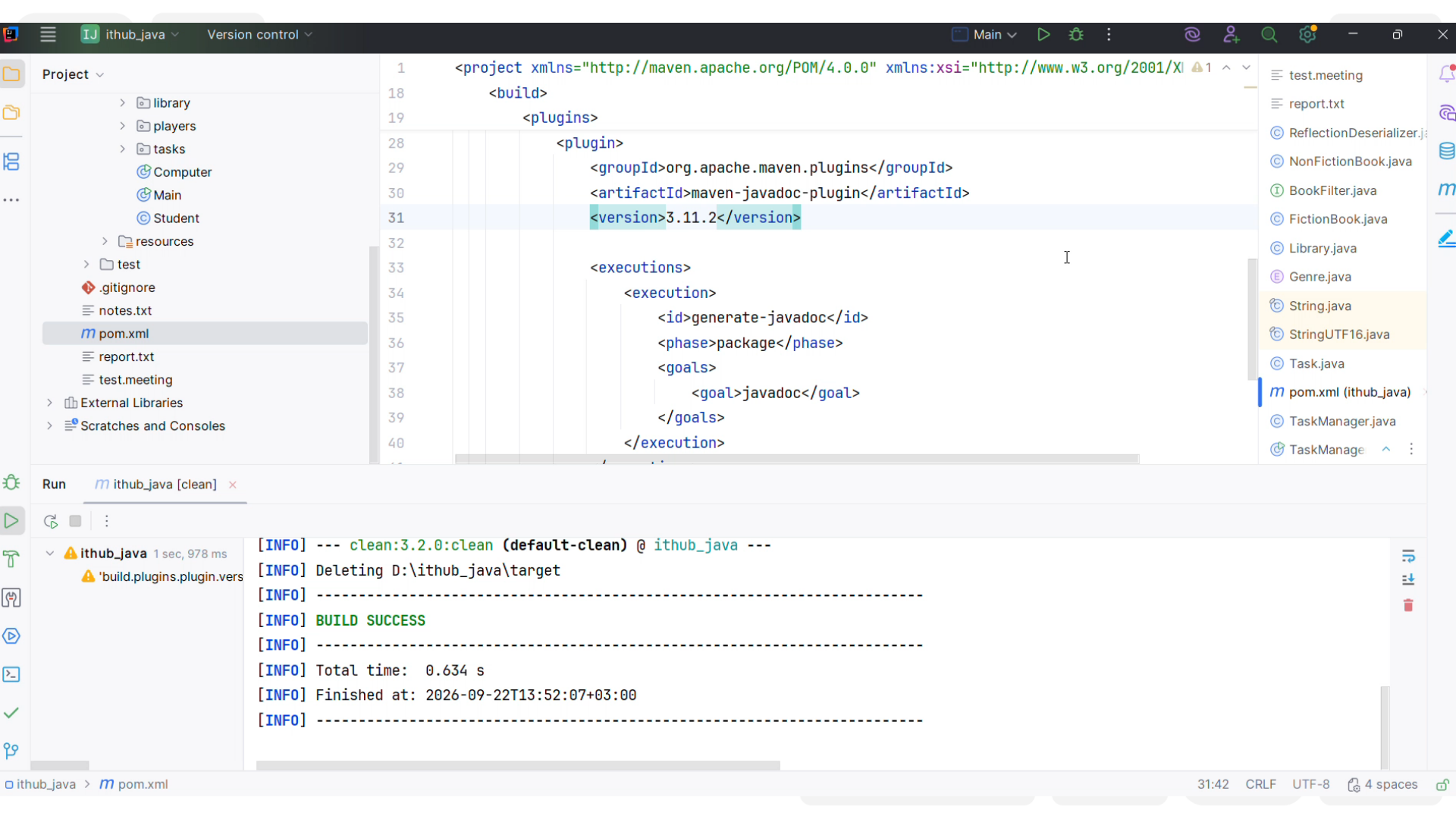

ithub_java
Version control
Main
Project
ithub_java D:\ithub_java
.idea
.mvn
data
src
main
java
annotations
calculate
files
kt5
library
players
tasks
Computer
Main
Student
resources
test
target
package tasks;
import java.time.LocalDate;
/**
* Создаёт новую задачу.
*
* <p>Имя задачи должно быть указано обязательно.
*/
public class Task 9 usages 2 inheritors
{
private String nameTask; 4 usages
private String opisTask; 4 usages
private String dateTask; 4 usages
public Task() { no usages
}
public Task(String nameTask, String opisTask, String dateTask) { 2 usages
this.nameTask = nameTask;
Meeting.java
ReflectionSerializer.java
kt5\Main.java
test.meeting
report.txt
ReflectionDeserializer.j
Book.java
NonFictionBook.java
BookFilter.java
FictionBook.java
Library.java
Genre.java
Readable.java
String.java
StringUTF16.java
Task.java
TaskManage
Terminal
Local
PS D:\ithub_java> javadoc -d docs src/main/java/tasks/*.java
ithub_java
docs
15:1
LF
UTF-8
4 spaces

```
<build>
   <plugins>
      <plugin>
         <groupId>org.apache.maven.plugins</groupId>
         <artifactId>maven-javadoc-plugin</artifactId>
         <version>3.11.2</version>

            <executions>
              <execution>
                 <id>generate-javadoc</id>
                 <phase>package</phase>
                 <goals>
                    <goal>javadoc</goal>
                 </goals>
              </execution>
            </executions>
      </plugin>
   </plugins>
</build>
```

IThub

Maven Javadoc Plugin позволяет создавать Javadoc непосредственно из Maven-проекта:


ithub_java
Version control
Main
Project
library
players
tasks
Computer
Main
Student
resources
test
.gitignore
notes.txt
pom.xml
report.txt
test.meeting
External Libraries
Scratches and Consoles
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/X
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-javadoc-plugin</artifactId>
<version>3.11.2</version>
<executions>
<execution>
<id>generate-javadoc</id>
<phase>package</phase>
<goals>
<goal>javadoc</goal>
</goals>
</execution>
test.meeting
report.txt
ReflectionDeserializer.j
NonFictionBook.java
BookFilter.java
FictionBook.java
Library.java
Genre.java
String.java
StringUTF16.java
Task.java
pom.xml (ithub_java)
TaskManager.java
TaskManage
Run
ithub_java [clean]
ithub_java 1 sec, 978 ms
'build.plugins.plugin.vers
[INFO] --- clean:3.2.0:clean (default-clean) @ ithub_java ---
[INFO] Deleting D:\ithub_java\target
[INFO] --
[INFO] BUILD SUCCESS
[INFO] --
[INFO] Total time: 0.634 s
[INFO] Finished at: 2026-09-22T13:52:07+03:00
[INFO] --
ithub_java
pom.xml
31:42
CRLF
UTF-8
4 spaces

Задача (КТ 5)

Программа для сериализации и десериализации данных.
Создайте консольное приложение, рядом с которым должна находиться директория data. На вход программе подаётся название файла, а также полное имя класса.
Пример класса:

```
public class Meeting {
 private String title;
 private String description;
 private Int usersLimit;
 private LocalDate startDate;
 private LocalTime startTimeFrom;
 private LocalTime startTimeTo;
 private LocalDateTime registrationAvailableUntil;
}
```

Задача (КТ 5)

Пример сериализованного объекта класса Meeting, данные которого были записаны в файл /data/test.meeting

```
'title'='Тестовое собрание'
'description'='Описание тестового собрания'
'usersLimit'='45'
'startDate'='20.08.2023'
'startTimeFrom'='10:00:00'
'startTimeTo'='18:00:00'
'registrationAvailableUntil'='19.08.2023 10:45:30'
```

Задача (КТ 5)

Для сериализации дат и времен используются следующие форматы:

Дата - 'День.Месяц.Год'
Время - 'Часы:Минуты:Секунды'
Дата и время - 'День.Месяц.Год Часы:Минуты:Секунды'

Задача (КТ 5)

IThub

- Необходимо реализовать класс **ReflectionSerializer**, с методом serialize, на вход которому подаётся Object object, а возвращать он должен String
  ReflectionSerializer должен преобразовать объект в текст, в соответствии с всеми правилами сериализации описанными в задании.

- Необходимо реализовать класс **ReflectionDeserializer**, с методом deserialize, на вход которому подаётся String serializedData и String className, а возвращать он должен Object
  ReflectionDeserializer должен преобразовать текст в объект необходимого класса.

Для тестирования реализуйте главный класс программы, в котором вы будете:
1) Загружать свой объект из файла
2) Изменять объект с помощью setter'ов
3) Сохранять свой объект в файл

ReflectionSerializer

Логика:

- Получаем поля класса ()
- Берём значения через Reflection.
- Определяем тип.
- Форматируем дату
  DateTimeFormatter.ofPattern
- Собираем строку.

```
Class<?> clazz = object.getClass()
Field[] fields = clazz.getDeclaredFields();
for(Field field : fields) {
    field.setAccessible(true);
    Object value = field.get(object);
…..
```

ReflectionDeserializer

Задача:

- получить класс по имени
- создать объект
- найти setter
- преобразовать строку в нужный тип

```
Class<?> clazz =
            Class.forName(className);

      Object object =
            clazz.getConstructor()
                  .newInstance();
…
Method setter =
                clazz.getMethod(….
```